



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Piotr
12.02.2025



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- **Summary of methodologies**

- Data Collection through API
- Data Collection with Web Scraping
- Data Wrangling - Exploratory Data Analysis with SQL
- Exploratory Data Analysis with Data Visualization
- Interactive Visual Analytics with Folium
- Machine Learning Prediction

- **Summary of all results**

- Exploratory Data Analysis result
- Interactive analytics in screenshots
- Predictive Analytics result

Introduction

Project Background and Context

SpaceX has dramatically altered the landscape of commercial space launches by advertising its Falcon 9 rocket launches at a significantly reduced price of around 62 million dollars on its website. In comparison, alternative providers typically charge upward of 165 million dollars per launch. A substantial portion of these cost savings is attributed to SpaceX's innovative ability to reuse the first stage of its rockets. Consequently, if we can develop a reliable method to predict whether the first stage will land successfully after launch, we can also more accurately estimate the overall cost of a launch. This predictive capability is not only valuable for internal cost optimization but can also serve as a strategic asset for any competing company interested in bidding against SpaceX for rocket launch contracts. The central objective of this project is to construct a comprehensive machine learning pipeline designed specifically to forecast the successful landing of the rocket's first stage. By achieving this, the pipeline will provide critical insights into cost management and competitive positioning in the highly dynamic space launch market.

Problems you want to find answers

- **Key Determinants of Landing Success:** What are the primary factors—ranging from technical specifications and operational parameters to environmental influences—that critically affect whether the rocket's first stage will land successfully?
- **Feature Interactions:** How do the various features and variables interact with each other, and what is the cumulative impact of these interactions on the success rate of the landing?
- **Operating Conditions:** What specific operating conditions need to be met or maintained to ensure the consistent and reliable success of the rocket landing program?

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - The dataset for this project was assembled by directly accessing the SpaceX API and by employing web scraping techniques to extract supplementary data from Wikipedia.
- Perform data wrangling
 - To prepare the categorical variables for analysis, we transformed them using one-hot encoding, converting each category into distinct binary features that can be easily processed by machine learning algorithms.
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Our approach for model development encompasses a full pipeline where we construct classification models, systematically tune their hyperparameters to optimize performance, and then rigorously evaluate their effectiveness using appropriate metrics and validation strategies.

Data Collection

- We collected the data using several complementary methods. Initially, we sent GET requests to the SpaceX API to retrieve the necessary information. The response was then decoded into JSON format using the `.json()` method and subsequently transformed into a pandas DataFrame via the `.json_normalize()` function. After this, we cleaned the dataset by identifying any missing values and filling them where appropriate. Additionally, we used BeautifulSoup to perform web scraping on Wikipedia pages containing Falcon 9 launch records. Our goal in that step was to extract the launch records presented as an HTML table, parse the table's contents, and convert it into another pandas DataFrame for further analysis.

Data Collection – SpaceX API

- We collected data from SpaceX by issuing a GET request to their API. Once we received the response, we converted it into JSON format, cleaned and performed basic data wrangling and formatting to prepare it for analysis. You can view all the details in our notebook at the following link:
- [IBM Data Science Capstone - Data Collection API.ipynb](#)

```
1. Get request for rocket launch data using API

In [6]: spacex_url="https://api.spacexdata.com/v4/launches/past"

In [7]: response = requests.get(spacex_url)

2. Use json_normalize method to convert json result to dataframe

In [12]: # Use json_normalize method to convert the json result into a dataframe
         # decode response content as json
         static_json_df = res.json()

In [13]: # apply json_normalize
         data = pd.json_normalize(static_json_df)

3. We then performed data cleaning and filling in the missing values

In [30]: rows = data_falcon9['PayloadMass'].values.tolist()[0]

         df_rows = pd.DataFrame(rows)
         df_rows = df_rows.replace(np.nan, PayloadMass)

         data_falcon9['PayloadMass'][0] = df_rows.values
         data_falcon9
```

Data Collection - Scraping

- We applied web scraping to collect Falcon 9 launch records by using BeautifulSoup. Specifically, we extracted the HTML table containing the launch data from the webpage, parsed the table, and then converted it into a pandas DataFrame for subsequent analysis. You can review the complete process in our notebook at the following link:
- [Data Collection with Web Scraping.ipynb](#)

```
1. Apply HTTP Get method to request the Falcon 9 rocket launch page

In [4]: static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"

In [5]: # use requests.get() method with the provided static_url
        # assign the response to a object
        html_data = requests.get(static_url)
        html_data.status_code

Out[5]: 200

2. Create a BeautifulSoup object from the HTML response

In [6]: # Use BeautifulSoup() to create a BeautifulSoup object from a response text content
        soup = BeautifulSoup(html_data.text, 'html.parser')

        Print the page title to verify if the BeautifulSoup object was created properly

In [7]: # Use soup.title attribute
        soup.title

Out[7]: <title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>

3. Extract all column names from the HTML table header

In [10]: column_names = []

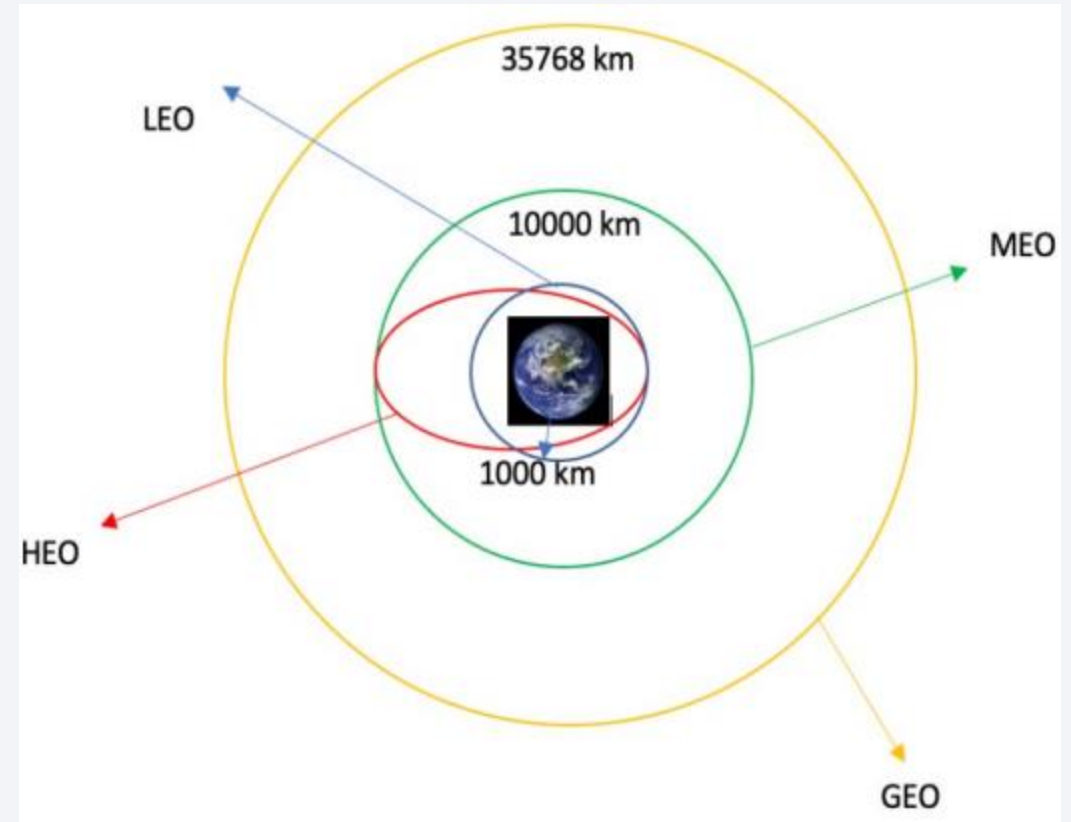
        # Apply find_all() function with 'th' element on first launch table
        # Iterate each th element and apply the provided extract_column_from_header() to get a column name
        # Append the Non-empty column name ('if name is not None and len(name) > 0') into a list called column_names

        element = soup.find_all('th')
        for row in range(len(element)):
            try:
                name = extract_column_from_header(element[row])
                if (name is not None and len(name) > 0):
                    column_names.append(name)
            except:
                pass

4. Create a dataframe by parsing the launch HTML tables
5. Export data to csv
```

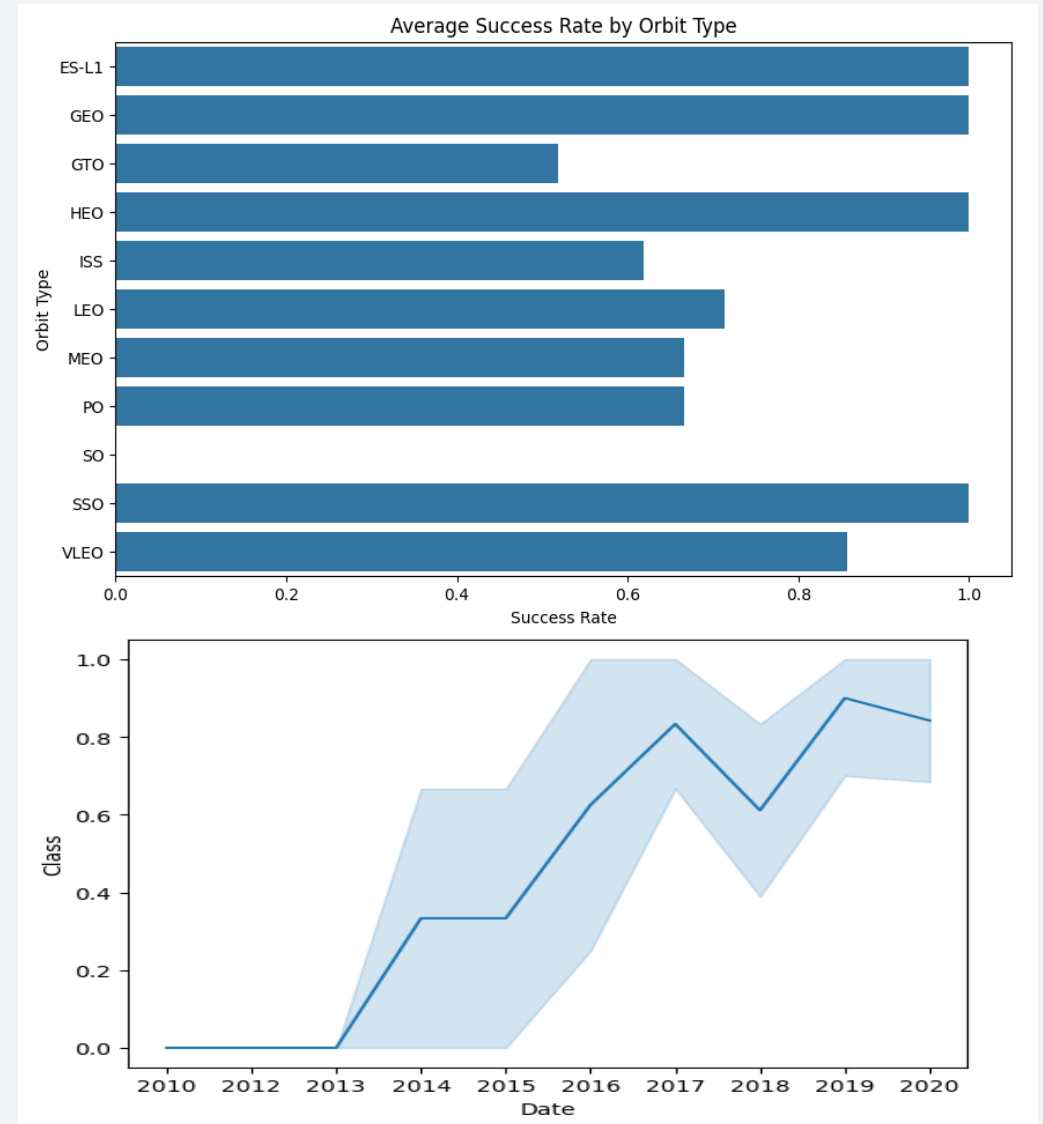
Data Wrangling

- We performed exploratory data analysis to determine the training labels for our model. In this process, we computed the number of launches for each site as well as the frequency and count of the different orbit types. Additionally, we generated a landing outcome label based on the existing outcome column and exported the finalized data to a CSV file for future use.
- For more details, please refer to notebook:
- [Data Wrangling.ipynb](#)



EDA with Data Visualization

- We conducted an exploratory analysis of the data by creating visualizations that illustrate several key relationships. Specifically, we examined how the flight number correlates with the launch site, how payload varies across different launch sites, and what the success rate is for each orbit type. In addition, we looked at the distribution of flight numbers by orbit type and tracked the yearly trend in launch success. For a detailed overview of our visualizations and findings, please refer to our notebook:
- [edadataviz.ipynb](#)



EDA with SQL

- We seamlessly integrated our data analysis workflow by loading the SpaceX dataset directly into a PostgreSQL database from within our Jupyter Notebook environment. Using SQL for exploratory data analysis, we crafted several queries to extract valuable insights from the data. For example, we identified the unique launch sites involved in the missions and computed the total payload mass carried by boosters launched under NASA's CRS contracts. We also determined the average payload mass for the Falcon 9 v1.1 booster version, as well as tallied the overall number of successful versus failed mission outcomes. Additionally, we isolated the records of failed landing attempts on drone ships, capturing details such as the booster version and the corresponding launch site. For a comprehensive look at our SQL queries and methodology, please refer to our notebook:
- https://github.com/piotersen/IBM_Data_Science_Capstone/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Build an Interactive Map with Folium

- We visualized the launch sites interactively using Folium. Specifically, we marked every launch site and enhanced the map with various objects—markers, circles, and lines—to indicate whether launches at each site were successful or not. Launch outcomes were encoded as binary classes (0 for failure and 1 for success), and we used color-coded marker clusters to easily identify sites with a relatively high success rate. Additionally, we calculated the distances from each launch site to nearby features (such as railways, highways, and coastlines) to investigate spatial relationships, for example, whether launch sites are located near major transportation routes or cities.
- For a comprehensive overview of our approach and visualizations, please refer to our notebook:
- [Interactive Visual Analytics with Folium.ipynb](#)

Build a Dashboard with Plotly Dash

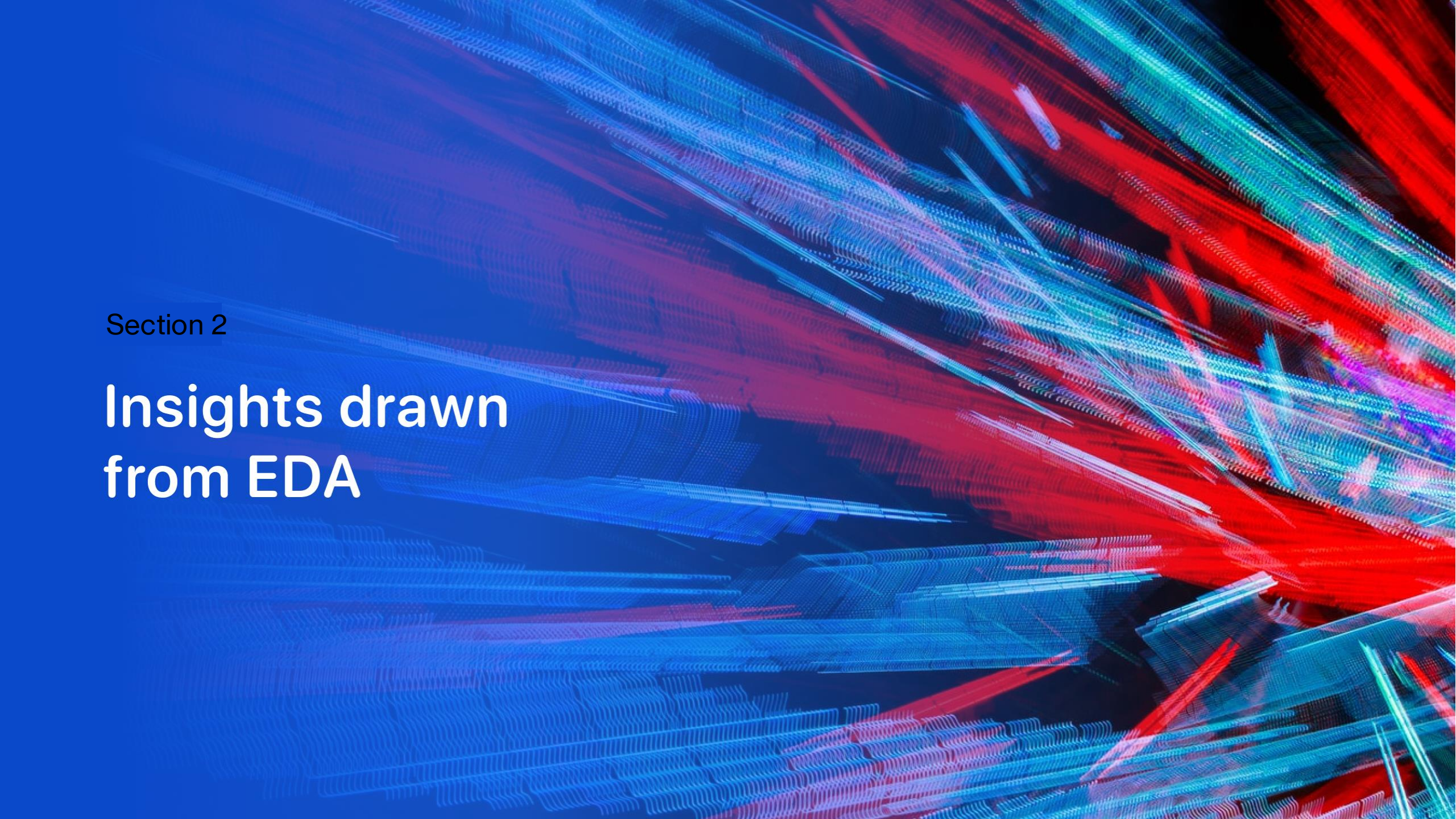
- We developed an interactive dashboard using Plotly Dash. In our dashboard, we created pie charts that display the distribution of total launches across various sites. Additionally, we built scatter plots that illustrate the relationship between the launch outcome and payload mass (in kg) for the different booster versions. For a detailed look at the code and implementation, please refer to our repository:
- [AppInPython.py](#)

Predictive Analysis (Classification)

- We began by loading our dataset using NumPy and pandas, performing the necessary data transformations, and splitting the data into training and testing sets. After that, we built a variety of machine learning models and optimized their hyperparameters with GridSearchCV, using accuracy as our primary evaluation metric. We further enhanced the model performance through feature engineering and algorithm tuning, ultimately identifying the best-performing classification model.
- For a detailed walkthrough of our process, please refer to our notebook:
- [Machine Learning Prediction.ipynb](#)

Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

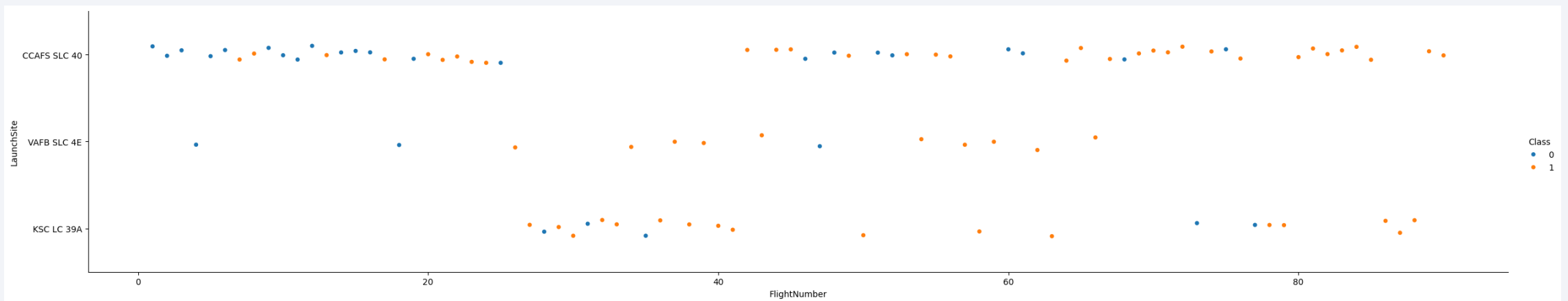
The background of the slide is an abstract composition. It features a solid blue area on the left side, which transitions into a dynamic pattern of diagonal streaks in shades of blue and red on the right. Overlaid on these streaks is a faint, semi-transparent grid pattern, giving the impression of a digital or data-driven environment.

Section 2

Insights drawn from EDA

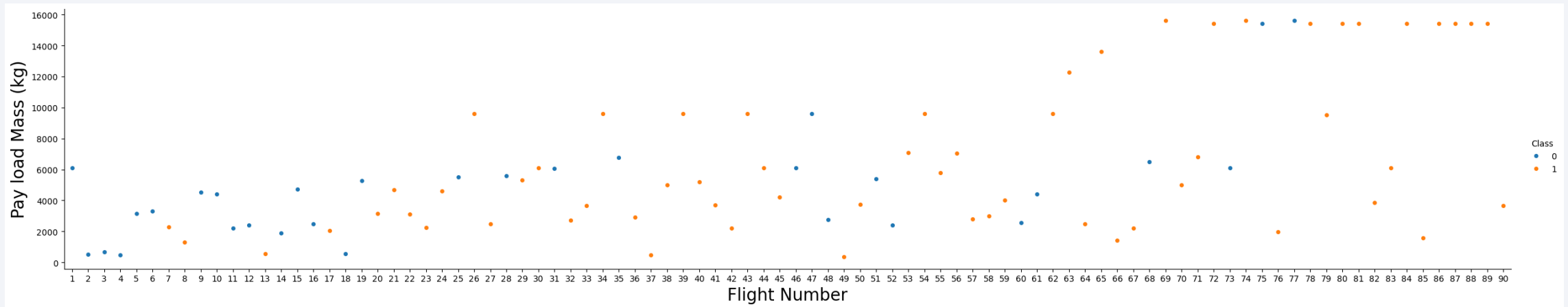
Flight Number vs. Launch Site

- Based on the visualization, we observed that launch sites with a higher number of flights generally exhibit a higher success rate. In other words, as the number of launches at a particular site increases, the probability of achieving a successful launch also tends to rise.



Payload vs. Launch Site

- The scatter plot visualizes the payload mass (kg) versus flight number, with different colors representing two classes (0 and 1). The x-axis represents flight numbers, while the y-axis shows the payload mass. The orange points (Class 1) and blue points (Class 0) indicate different categories of flights, likely distinguishing successful and unsuccessful missions. The trend suggests that later flights carry heavier payloads, especially among successful launches.

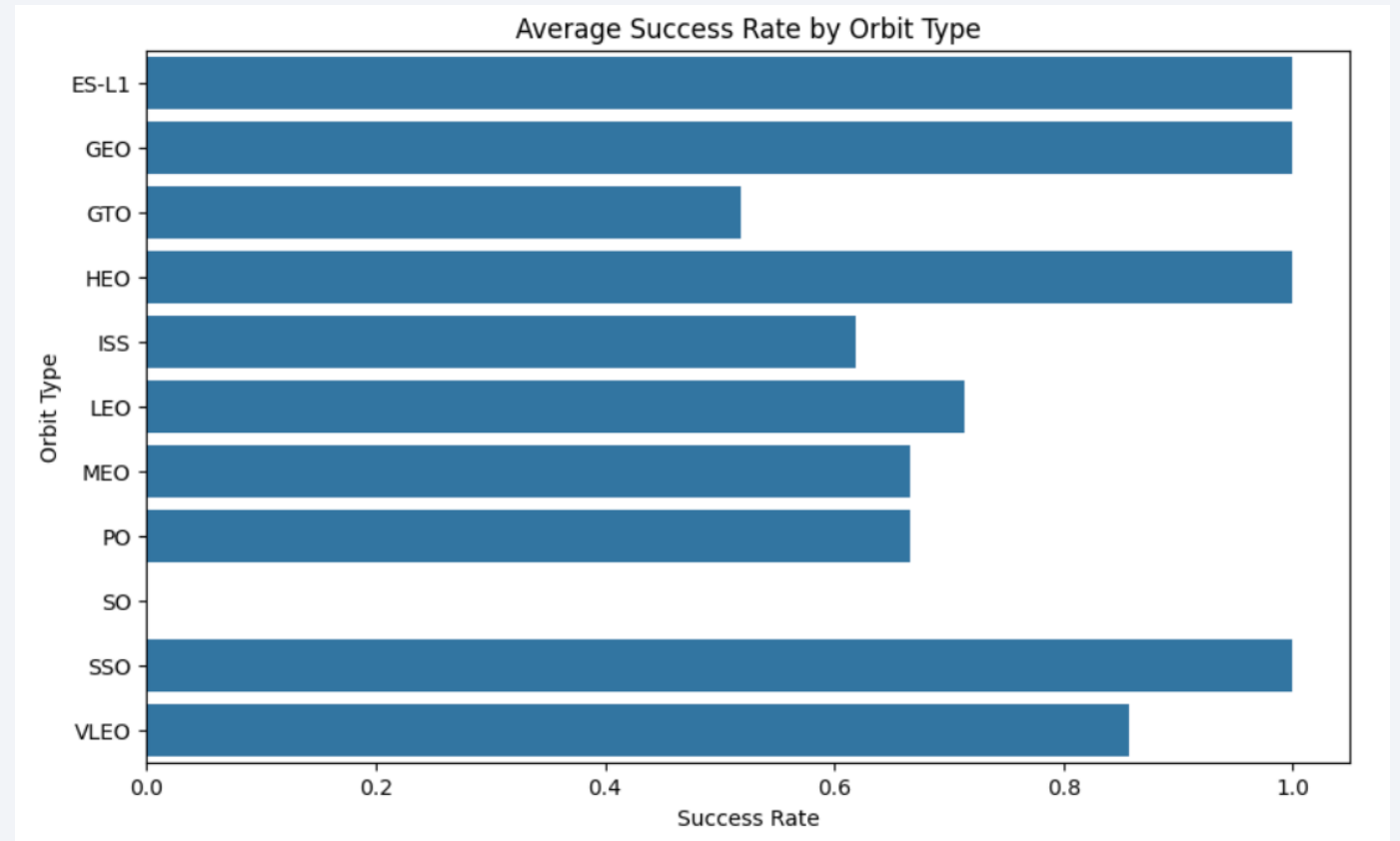


Success Rate vs. Orbit Type

The bar chart displays the **average success rate of space launches categorized by orbit type**. The x-axis represents the **success rate**, while the y-axis lists various **orbit types** (e.g., GEO, LEO, SSO, ISS). The lengths of the bars indicate the relative success rates for each orbit type.

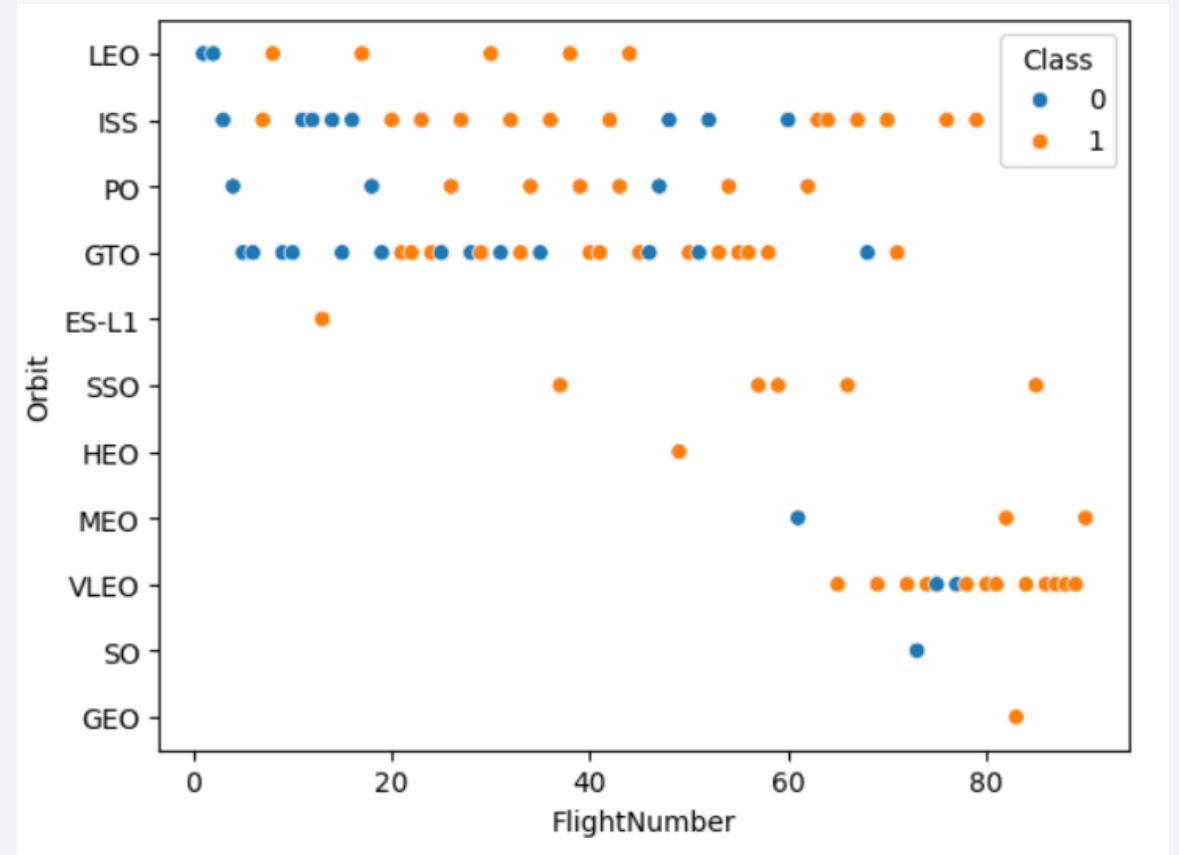
Key observations:

- Some orbit types, like **ES-L1, GEO, HEO, and SSO**, show **very high success rates** close to 1.0.
- Other orbit types, such as **GTO and ISS**, have **lower success rates** compared to the rest.
- The **LEO and VLEO** orbits also have relatively high success rates.
- This chart helps in identifying which orbit types tend to have more reliable launches.



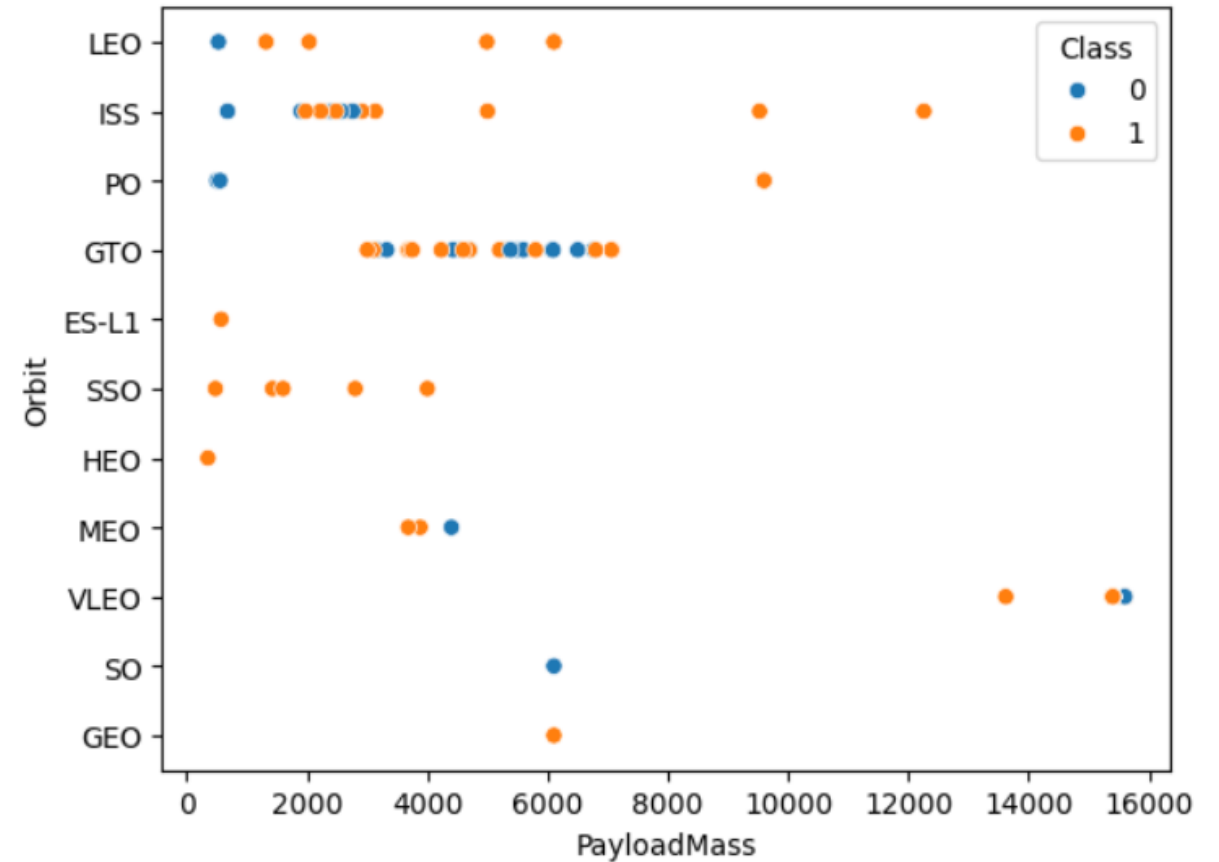
Flight Number vs. Orbit Type

- The plot comparing Flight Number against Orbit Type reveals a distinct pattern for LEO missions—launch sites that have conducted a higher number of flights tend to have a higher success rate in Low Earth Orbit. In contrast, for launches targeting Geostationary Transfer Orbit (GTO), no clear relationship is observed between the number of flights and the success rate. This suggests that while experience (as indicated by more flights) plays a role in achieving success in LEO, other factors likely influence the outcomes for GTO missions.



Payload vs. Orbit Type

- We can observe that for launches with heavy payloads, the rate of successful landings is notably higher when targeting orbits such as PO, LEO, and ISS. This indicates that these specific orbits tend to have better outcomes in terms of landing success under conditions involving heavier payloads.

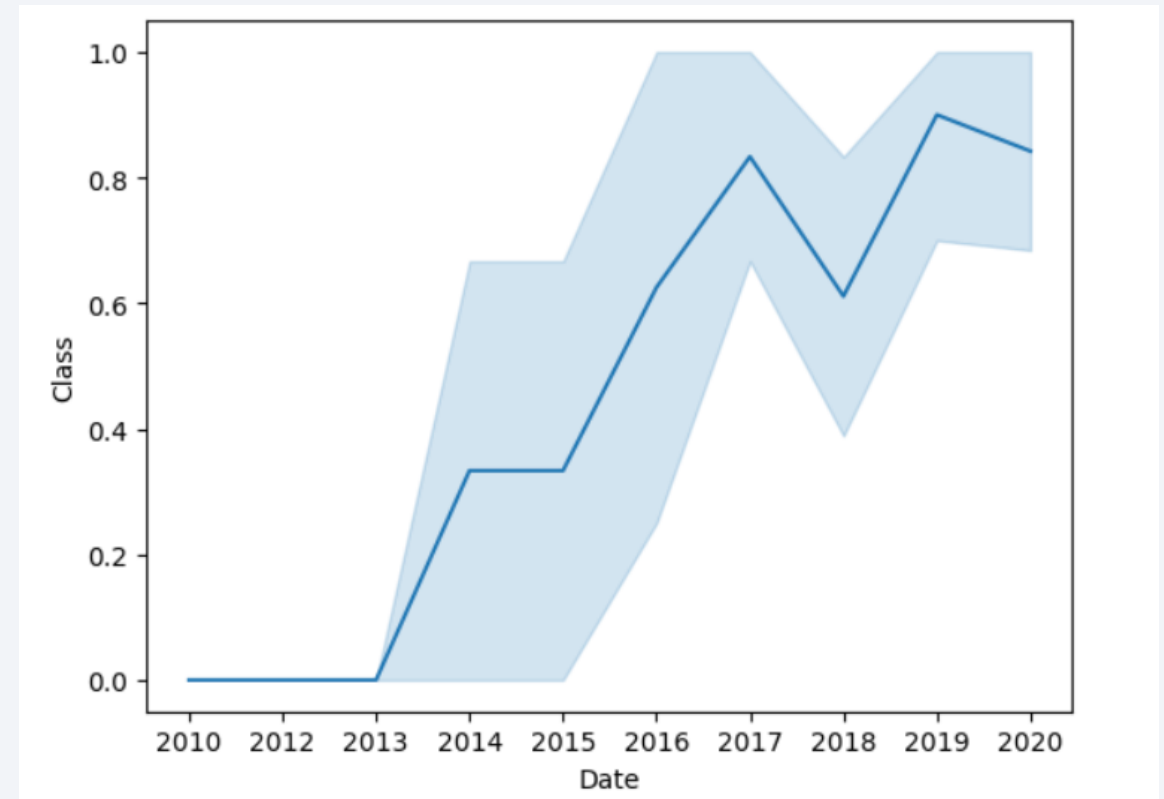


Launch Success Yearly Trend

The line plot illustrates the **trend of a "Class" variable over time (from 2010 to 2020)**, with a shaded region representing some level of uncertainty or variation (likely a confidence interval).

Key Observations:

- **Before 2013**, the Class value remains **at 0**, suggesting no successful occurrences.
- From **2013 onward**, the Class value **gradually increases**, indicating an **improvement in success rates** over time.
- The **peak is reached around 2017-2019**, where the Class value is close to **1**, signifying a high success rate.
- There is some fluctuation, but overall, the trend suggests **a steady increase in performance**.
- The **shaded region** suggests variance, which **narrows over time**, indicating improved consistency in success rates.
- This visualization likely tracks **launch success rates or reliability over time**, showing significant progress after 2013.



All Launch Site Names

- We employed the DISTINCT keyword in our SQL queries to filter out duplicate entries and display only the unique launch sites from the SpaceX dataset.

Display the names of the unique launch sites in the space mission

```
In [10]: task_1 = '''  
          SELECT DISTINCT LaunchSite  
          FROM SpaceX  
          ...  
          create_pandas_df(task_1, database=conn)
```

```
Out[10]:
```

	launchsite
0	KSC LC-39A
1	CCAFS LC-40
2	CCAFS SLC-40
3	VAFB SLC-4E

Launch Site Names Begin with 'CCA'

- We executed the query to retrieve and display five records where the launch site names begin with “CCA.”

Display 5 records where launch sites begin with the string 'CCA'

```
In [14]: %sql select * from SPACEXTBL Where Launch_Site LIKE "CCA%" LIMIT 5
```

* sqlite:///my_data1.db
Done.

Out[14]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (p
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (p
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	N
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	N
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	N

Total Payload Mass

- We executed the following SQL query to compute the aggregate payload carried by boosters associated with NASA. The query returned a total payload value of 45596.

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
In [15]: %sql select sum(PAYLOAD_MASS__KG_) from SPACEXTBL WHERE Customer = "NASA (CRS)"
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[15]: sum(PAYLOAD_MASS__KG_)  
          45596
```

Average Payload Mass by F9 v1.1

- We calculated the average payload mass for the F9 v1.1 booster as 2928.4 by filtering the dataset for that booster version and applying an AVG function in SQL. This value provides a quick insight into the typical payload capacity of the F9 v1.1.

Task 4

Display average payload mass carried by booster version F9 v1.1

```
In [16]: %sql select round(avg(PAYLOAD_MASS__KG_)) from SPACEXTBL WHERE Booster_Version LIKE "F9 v1.1%"
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[16]: round(avg(PAYLOAD_MASS__KG_))  
                2535.0
```

First Successful Ground Landing Date

- We observed that the first successful landing on a ground pad occurred on December 22, 2015.

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
In [17]: %sql select min(Date) from SPACEXTBL WHERE Landing_Outcome LIKE "Success (ground pad)"
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[17]: min(Date)  
2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- We filtered the dataset using a WHERE clause to select boosters that successfully landed on a drone ship, and then applied an AND condition to further narrow down the records to those with a payload mass between 4000 and 6000.

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

In [18]: `%sql select * from SPACEXTBL WHERE Landing_Outcome LIKE "Success (drone ship)" AND PAYLOAD_MASS_KG_ BETWEEN 4000`

* sqlite:///my_data1.db
Done.

Out[18]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
------	------------	-----------------	-------------	---------	------------------	-------	----------	-----------------	-----------------

016-05-06	5:21:00	F9 FT B1022	CCAFS LC-40	JCSAT-14	4696	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
-----------	---------	-------------	-------------	----------	------	-----	------------------------	---------	----------------------

016-08-14	5:26:00	F9 FT B1026	CCAFS LC-40	JCSAT-16	4600	GTO	SKY Perfect JSAT Group	Success	Success (drone ship)
-----------	---------	-------------	-------------	----------	------	-----	------------------------	---------	----------------------

017-03-30	22:27:00	F9 FT B1021.2	KSC LC-39A	SES-10	5300	GTO	SES	Success	Success (drone ship)
-----------	----------	---------------	------------	--------	------	-----	-----	---------	----------------------

017-0-11	22:53:00	F9 FT B1031.2	KSC LC-39A	SES-11 / EchoStar 105	5200	GTO	SES EchoStar	Success	Success (drone ship)
----------	----------	---------------	------------	-----------------------	------	-----	--------------	---------	----------------------

Total Number of Successful and Failure Mission Outcomes

- We employed the SQL wildcard character '%' in the WHERE clause to filter the MissionOutcome field for records that contained either a success or a failure.

List the total number of successful and failure mission outcomes

```
In [16]: task_7a = '''
          SELECT COUNT(MissionOutcome) AS SuccessOutcome
          FROM SpaceX
          WHERE MissionOutcome LIKE 'Success%'
          '''

          task_7b = '''
          SELECT COUNT(MissionOutcome) AS FailureOutcome
          FROM SpaceX
          WHERE MissionOutcome LIKE 'Failure%'
          '''

          print('The total number of successful mission outcome is:')
          display(create_pandas_df(task_7a, database=conn))
          print()
          print('The total number of failed mission outcome is:')
          create_pandas_df(task_7b, database=conn)
```

The total number of successful mission outcome is:

	successoutcome
0	100

The total number of failed mission outcome is:

```
Out[16]:
```

	failureoutcome
0	1

Boosters Carried Maximum Payload

- We determined the booster that carried the maximum payload by using a subquery within the WHERE clause along with the MAX() function. The subquery identified the highest payload value, and we then selected the booster record matching this value.

Task 8

List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

```
In [32]: %sql select Booster_Version from SPACEXTBL where PAYLOAD_MASS_KG_ = (select max(PAYLOAD_MASS_KG_) from SPACEXTBL)

* sqlite:///my_data1.db
Done.
```

Out[32]: **Booster_Version**

F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

- We used a combination of SQL conditions—including WHERE with LIKE, AND, and BETWEEN—to filter for records from 2015 that had failed landing outcomes on a drone ship. This query allowed us to retrieve the associated booster versions and launch site names for those failures.

Task 9

List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.

Note: SQLite does not support monthnames. So you need to use substr(Date, 6,2) as month to get the months and substr(Date,0,5)='2015' for year.

In [45]: `%sql SELECT substr(Date, 6, 2) AS month, landing_outcome, Booster_Version, launch_site FROM SPACEXTBL WHERE subst`

* sqlite:///my_data1.db
Done.

Out[45]:

	month	Landing_Outcome	Booster_Version	Launch_Site
	01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
	04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- We selected the landing outcomes along with their corresponding counts from the dataset. Then, using a WHERE clause, we filtered the data for landing outcomes that occurred between 2010-06-04 and 2010-03-20. After that, we grouped the results by the landing outcomes using the GROUP BY clause and finally ordered the groups in descending order using the ORDER BY clause.

Task 10

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

In [49]: `%sql SELECT Landing_Outcome,COUNT(*) AS outcome_count,RANK() OVER (ORDER BY COUNT(*) DESC) AS ranking FROM SPACE`

* sqlite:///my_data1.db
Done.

Out[49]:

Landing_Outcome	outcome_count	ranking
No attempt	10	1
Success (drone ship)	5	2
Failure (drone ship)	5	2
Success (ground pad)	3	4
Controlled (ocean)	3	4
Uncontrolled (ocean)	2	6
Failure (parachute)	2	6
Precluded (drone ship)	1	8

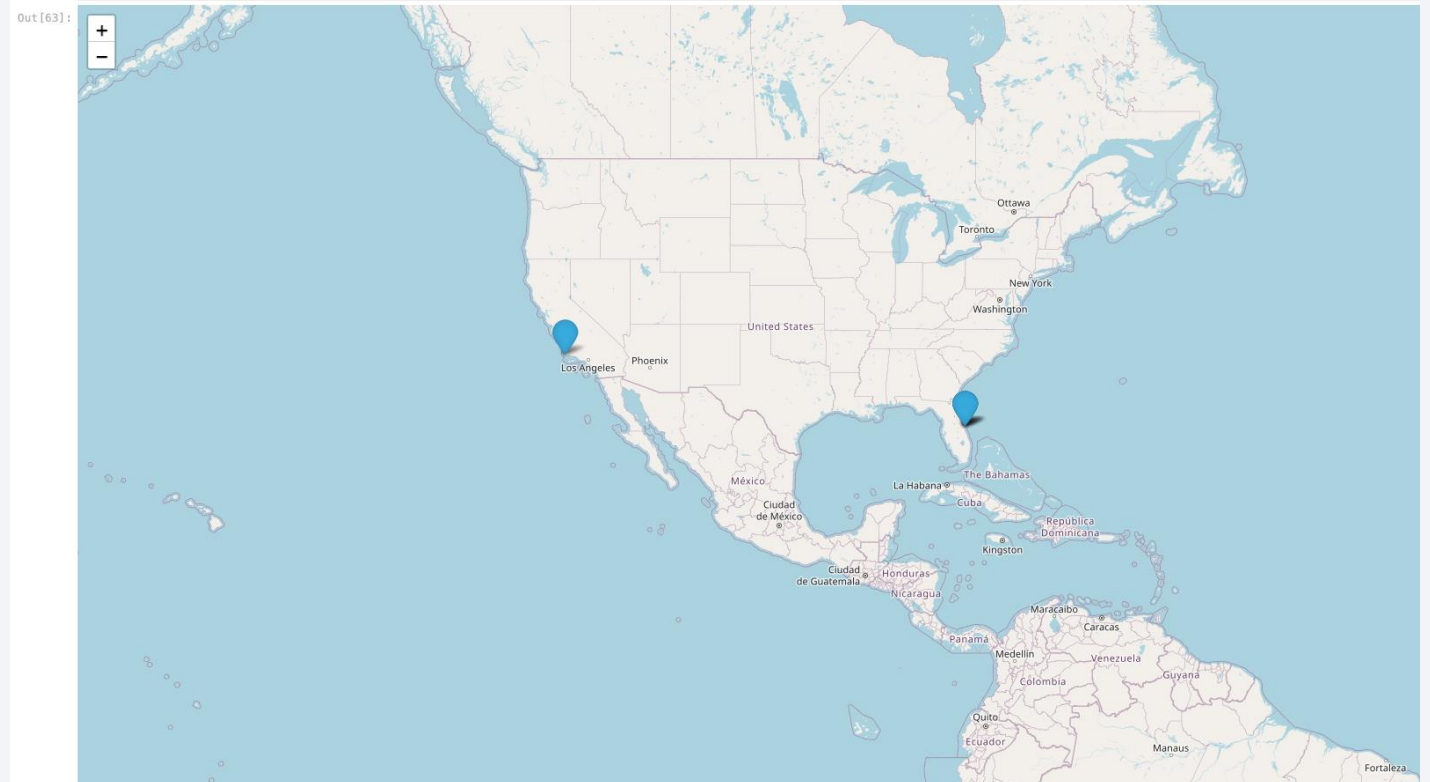
Section 3

Launch Sites Proximities Analysis

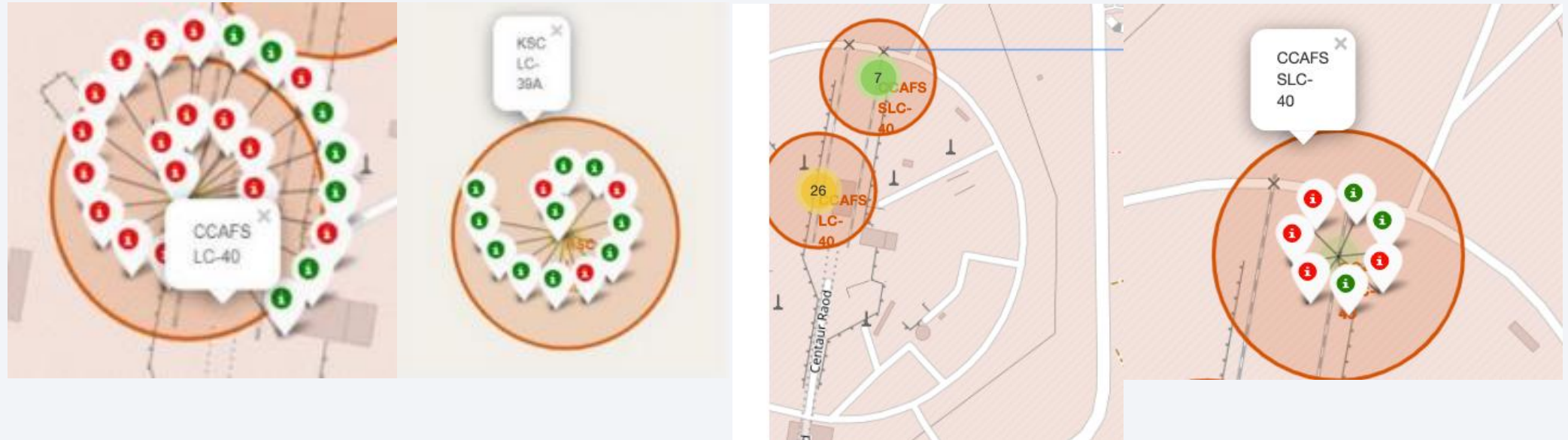


<Folium Map Screenshot 1>

- Our analysis reveals that SpaceX's launch sites are predominantly located along the coasts of the United States—specifically in Florida and California. This strategic placement supports their operational needs, such as utilizing drone ships for landings and taking advantage of coastal launch trajectories.

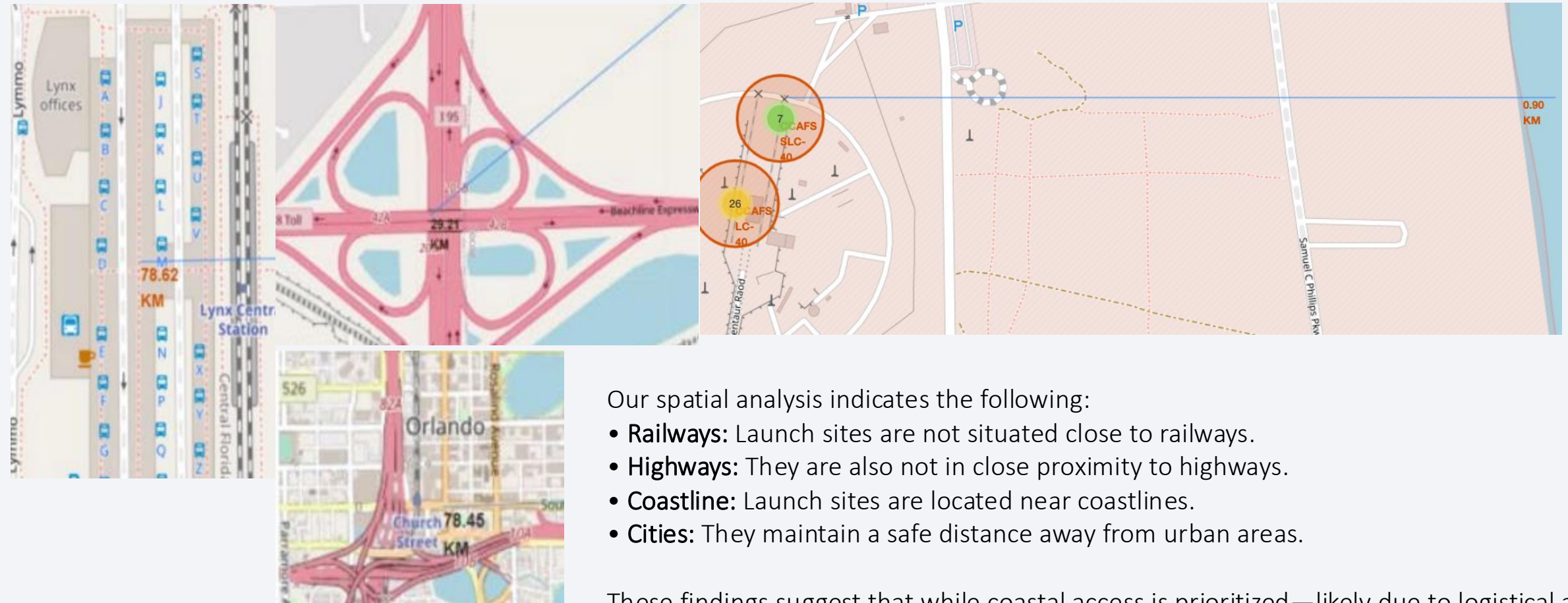


Markers showing launch sites with color labels



- In our visualization of Florida launch sites, we used color-coded markers—green for successful launches and red for failed ones—to clearly indicate the outcomes at each site.

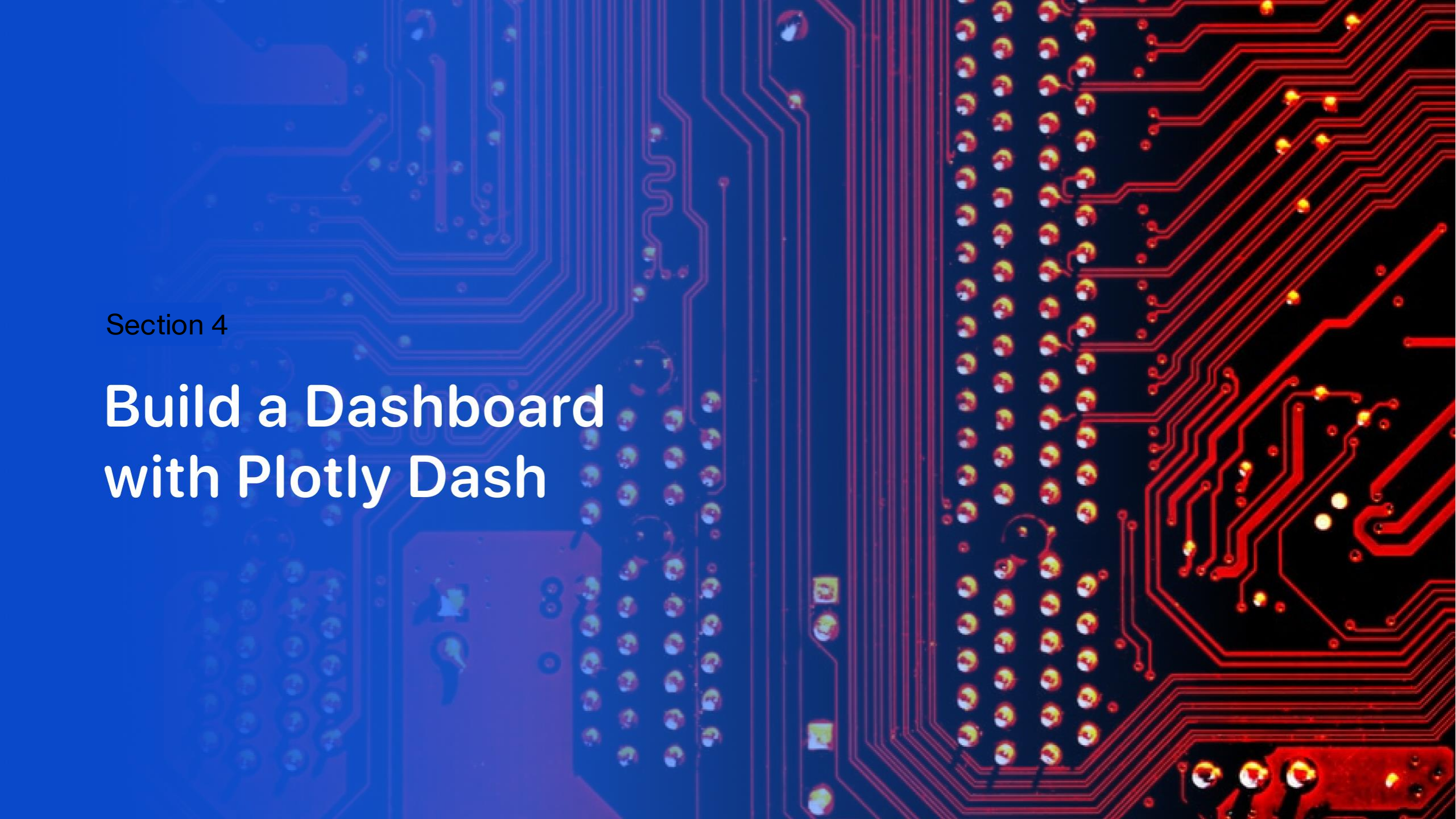
Launch Site distance to landmarks



Our spatial analysis indicates the following:

- **Railways:** Launch sites are not situated close to railways.
- **Highways:** They are also not in close proximity to highways.
- **Coastline:** Launch sites are located near coastlines.
- **Cities:** They maintain a safe distance away from urban areas.

These findings suggest that while coastal access is prioritized—likely due to logistical and operational benefits such as drone ship recoveries—other infrastructure like railways and highways are not major factors in the site selection, and there is a deliberate effort to keep the launch sites away from densely populated areas for safety reasons.

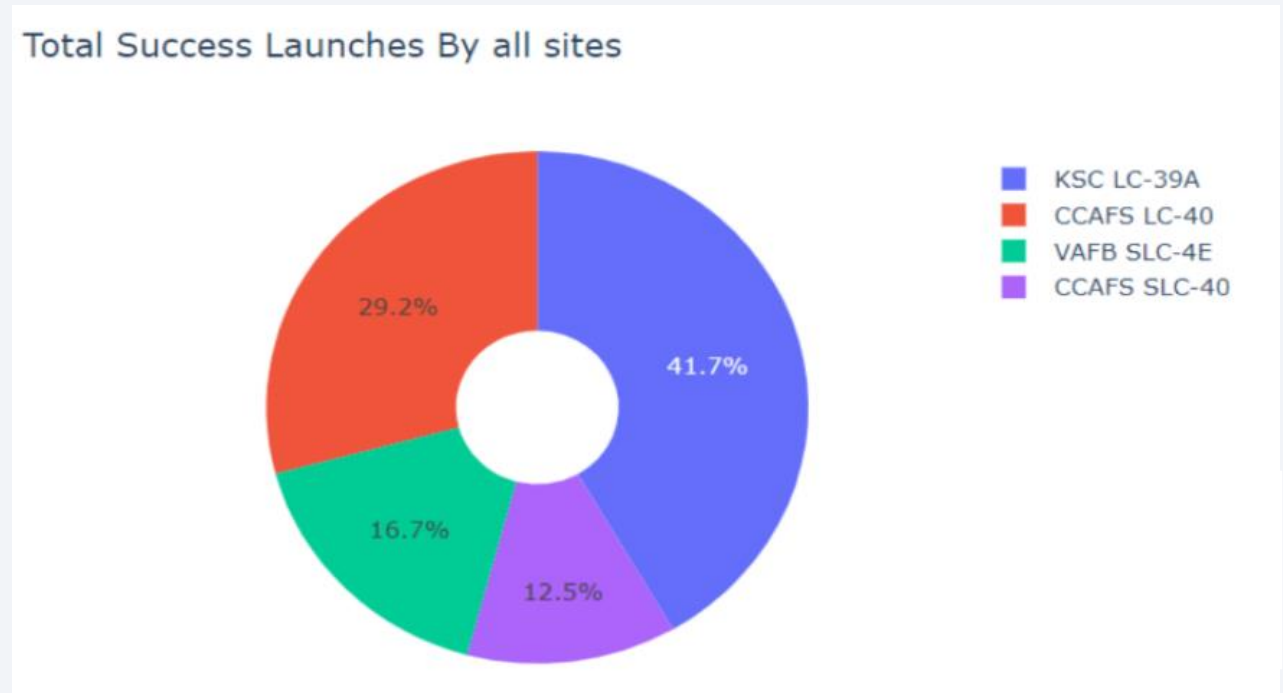


Section 4

Build a Dashboard with Plotly Dash

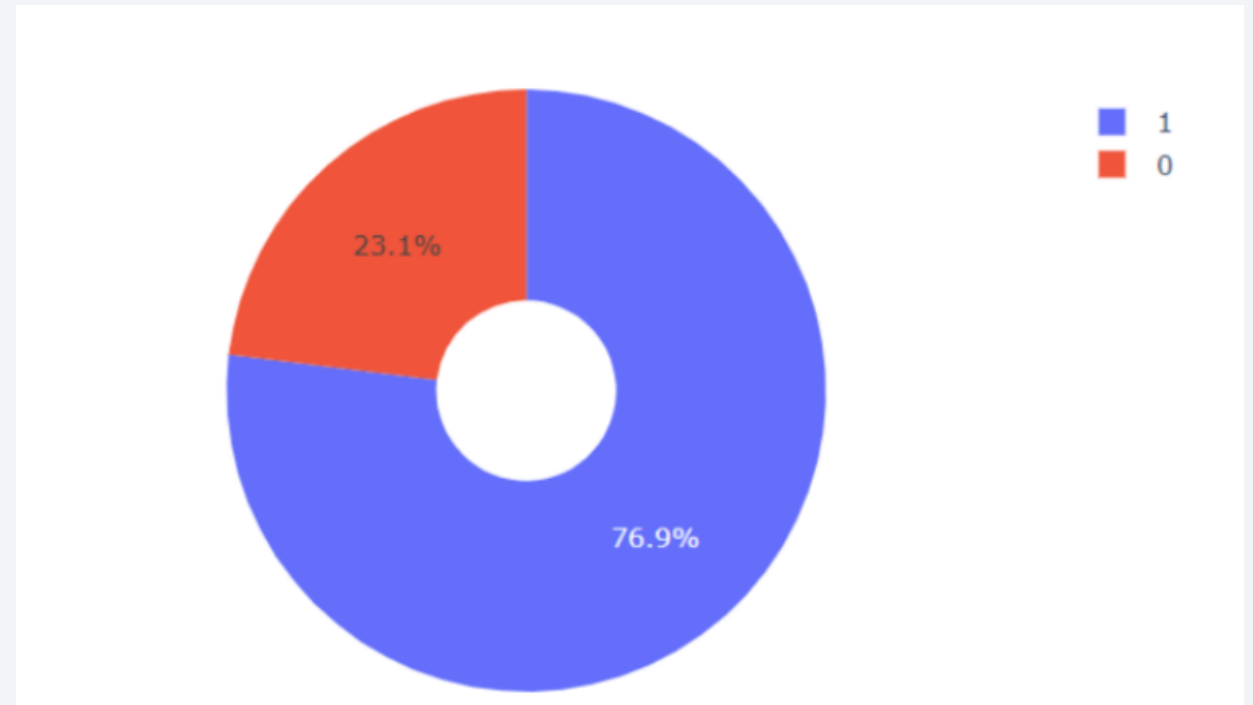
The pie chart visualizes the success percentage for each launch site

The pie chart visualizes the success percentage for each launch site, and it clearly indicates that KSC LC-39A recorded the highest success rate among all the sites.

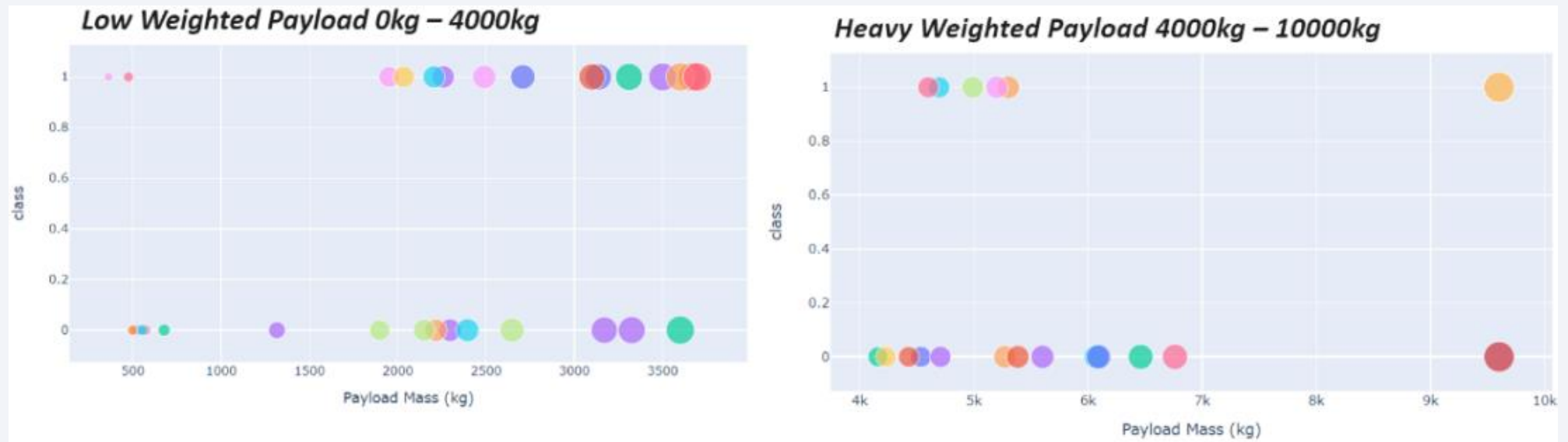


KSC LC-39A Launch Success Ratio – 76.9% Success, 23.1% Failure

KSC LC-39A achieved a success rate of 76.9%, with a corresponding failure rate of 23.1%. This means that approximately three out of every four launches from KSC LC-39A have been successful, while one out of every four has resulted in a failure.



This scatter plot displays payload versus launch outcome for all sites, with a slider to adjust the payload range.



Scatter plot displaying payload versus launch outcome for all sites, with a slider to select the payload range.



Section 5

Predictive Analysis (Classification)

Classification Accuracy

```
models = {'KNeighbors': knn_cv.best_score_,
          'DecisionTree': tree_cv.best_score_,
          'LogisticRegression': logreg_cv.best_score_,
          'SupportVector': svm_cv.best_score_}

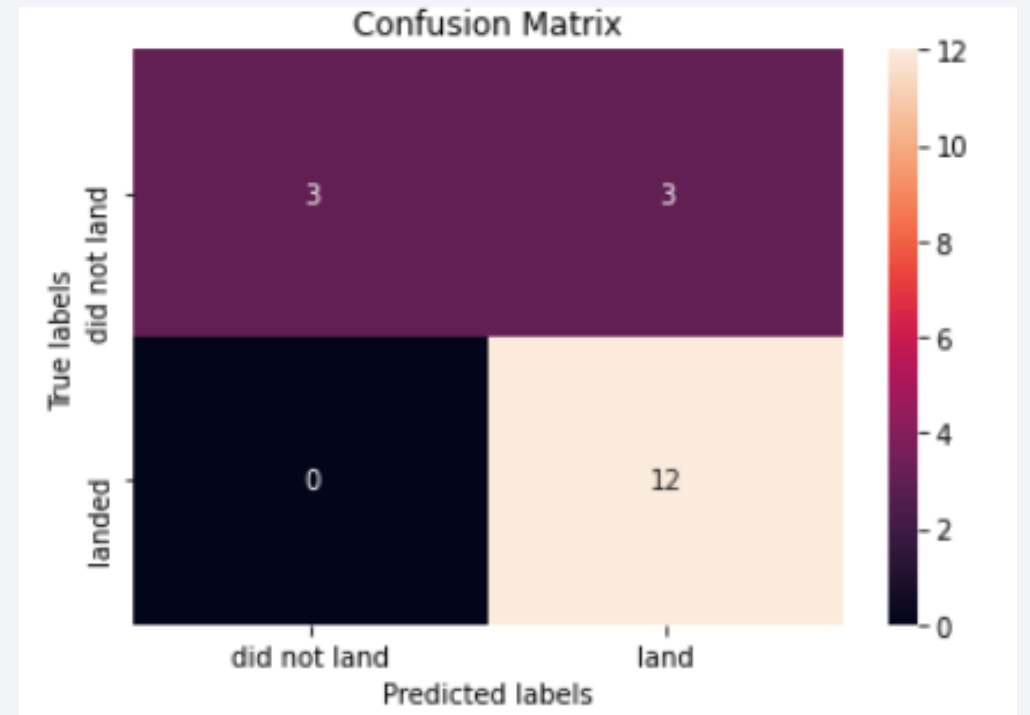
bestalgorithm = max(models, key=models.get)
print('Best model is', bestalgorithm, 'with a score of', models[bestalgorithm])
if bestalgorithm == 'DecisionTree':
    print('Best params is :', tree_cv.best_params_)
if bestalgorithm == 'KNeighbors':
    print('Best params is :', knn_cv.best_params_)
if bestalgorithm == 'LogisticRegression':
    print('Best params is :', logreg_cv.best_params_)
if bestalgorithm == 'SupportVector':
    print('Best params is :', svm_cv.best_params_)
```

Best model is DecisionTree with a score of 0.8732142857142856

Best params is : {'criterion': 'gini', 'max_depth': 6, 'max_features': 'auto', 'min_samples_leaf': 2, 'min_samples_split': 5, 'splitter': 'random'}

Confusion Matrix

- The confusion matrix for the decision tree classifier indicates that while it effectively differentiates between classes, its primary issue is a high rate of false positives—unsuccessful landings being misclassified as successful.



Conclusions

Based on our analysis, we can conclude the following:

- **Flight Volume and Success:** There is a strong positive correlation between the number of flights at a launch site and its success rate. In other words, the more flights conducted from a particular site, the higher the likelihood of a successful launch, suggesting that accumulated operational experience plays a crucial role in achieving positive outcomes.
- **Trends in Launch Success:** The data shows that the success rate of launches steadily increased from 2013 through 2020. This upward trend likely reflects ongoing technological advancements and improvements in operational procedures over time.
- **Orbital Success Rates:** When examining different orbital trajectories, we found that orbits such as ES-L1, GEO, HEO, SSO, and VLEO exhibit the highest success rates. These results indicate that certain orbital paths may offer more favorable conditions for successful launches.
- **Outstanding Launch Site:** Among all the launch sites analyzed, KSC LC-39A stands out with the highest number of successful launches. This highlights its critical role in SpaceX's operations and its effectiveness as a launch site.
- **Machine Learning Model Performance:** In our predictive modeling efforts, the decision tree classifier emerged as the best machine learning algorithm for this task. It demonstrated superior performance in distinguishing between successful and unsuccessful landing outcomes.

These conclusions not only highlight the importance of operational experience and technological progress but also provide valuable insights into which factors and conditions contribute most significantly to the success of SpaceX launches.

Thank you!

